

Coding & Solution

Date	15 March 2026
Team ID	
Project Name	SmartLender - Applicant Credibility Prediction for Loan Approval
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/AP24110010056/Smart-Lender
Programming Language(s)	Python
Framework(s) Used	Flask (backend), scikit-learn (ML model), deployed with Gunicorn
Key Features Implemented	Loan eligibility prediction using a trained Random Forest model; web form for applicant data entry; feature scaling; instant result display.
Pending / Incomplete Features	Input validation/error handling for invalid form data; user authentication; database storage of past predictions.
Setup / Run Instructions	1) pip install -r requirements.txt 2) Ensure rdf.pkl and scale1.pkl are in the Flask folder 3) Run 'python app.py' 4) Open the local server URL in a browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	No
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial
4	Error handling is implemented for critical operations	No
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

Core prediction flow (data entry, scaling, Random Forest inference, result display) works end-to-end and is deployed. Main gaps are the lack of try/except error handling around form input and the absence of a modular code structure (all logic currently lives inside a single Flask route).